29/04/2026

ELE432 ADVANCED DIGITAL DESIGN

PRELIMINARY WORK 3 (LABORATORY-3 ASSIGNMENT)

ASLI NUR YEŞİLTAŞ 2200357018

1)Introduction

This lab presents the design and verification of a multicycle RISC-V processor capable of executing a subset of the RV32I instruction set, specifically `addi`, `or`, `and`, `add`, `sub`, `slt`, `beq`, `lw`, `sw`, and `jal`. The microarchitecture follows the multicycle organization described in Chapter 7.4 of *Digital Design and Computer Architecture: RISC-V Edition* by Harris and Harris, and is consistent with the block diagram given in Figure 2 of the lab handout.

In contrast to the single-cycle processor studied previously, the multicycle processor uses a single unified memory for both instructions and data, and divides the execution of each instruction into a sequence of shorter steps controlled by a finite state machine. Branch and store instructions complete in three to four cycles, R-type and I-type ALU instructions in four cycles, and load instructions in five cycles. This trades a higher number of cycles per instruction for a shorter critical path and a smaller hardware footprint, since a single ALU and a single memory port are reused across the steps of every instruction.

The processor is described in SystemVerilog as a hierarchy of three primary modules: a `top` module that wires together the processor core (`riscv`) and a unified `memory` module. The `riscv` core is itself decomposed into a `controller` (containing the Main FSM, the ALU Decoder, and the Instruction Decoder) and a `datapath` built from generic, parameterized building blocks. Functional correctness was verified using the testbench (`riscv_testbench.sv`) and test program (`riscvtest.s`) supplied with the lab. The remainder of this report documents the module hierarchy, the top-level interface, the verification methodology, the predicted operation of the first three instructions, and the resulting simulation waveforms.

2) Module hierarchy and inter-module signals

The top-level organization of the multicycle processor is shown in Figure 1. The design is partitioned into three primary modules, consistent with Figure 7.27 of the textbook and Figure 2 of the lab handout:

- The riscv module forms the processor core and is composed of two submodules: controller (an 11-state Moore machine that contains the Main FSM, ALU Decoder, and Instruction Decoder) and datapath (which holds the PC, OldPC, IR, Data, A, and ALUOut registers along with the Register File, sign-extension unit, and ALU).

- The memory module implements the unified instruction/data memory required by the multicycle architecture; its initial contents are loaded from memfile.txt via $readmemh.

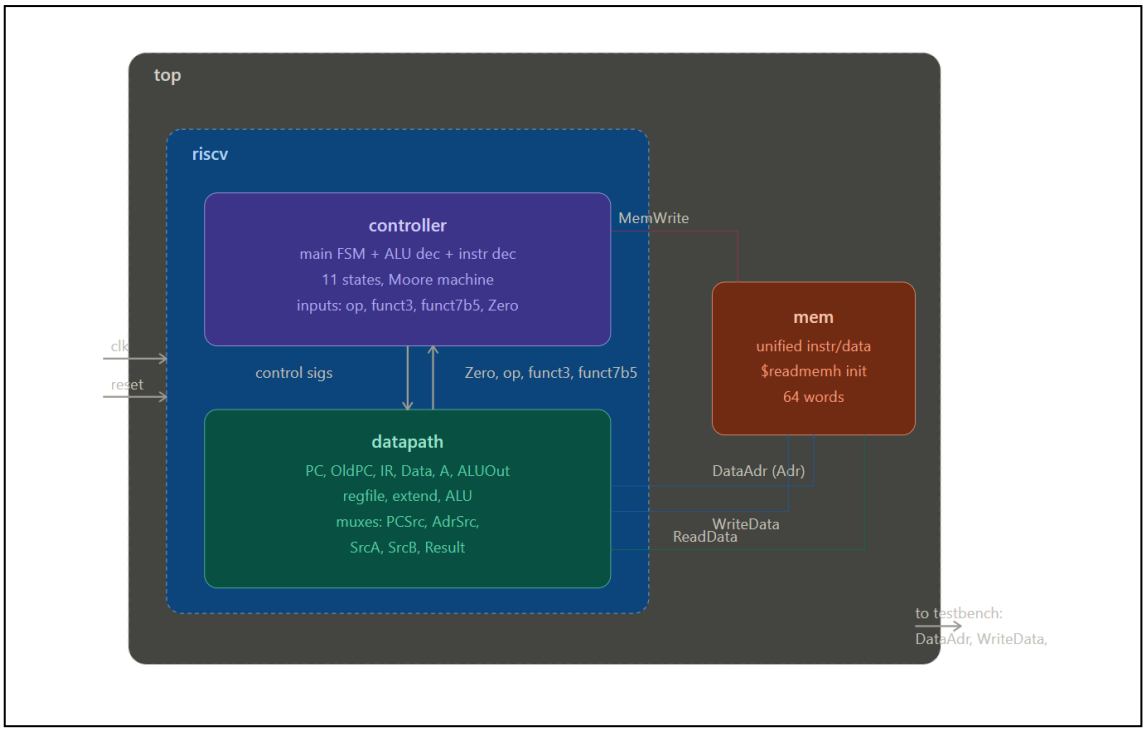

Figure 1. Module hierarchy and inter-module signals

(Hierarchical organization of the top, riscv (controller + datapath), and memory modules)

- The top module wires riscv and memory together and exposes only the three signals that the supplied testbench observes: DataAdr, WriteData, and MemWrite.

Communication between the controller and the datapath is bidirectional. The controller drives the datapath with control signals (PCWrite, AdrSrc, IRWrite, RegWrite, ResultSrc, ALUSrcA, ALUSrcB, ALUControl, and ImmSrc), while the datapath feeds back the Zero flag together with the instruction fields op[6:0], funct3[2:0], and funct7b5 so that the controller can decode the current instruction. The interface between riscv and memory is a standard memory bus: the address (DataAdr) and write data (WriteData) originate in the datapath, the write enable (MemWrite) is generated by the controller, and the read data (ReadData) is returned asynchronously to the datapath.

3) Top-level module declaration

The top-level module's port list matches the declaration required by the lab handout exactly, so that it can be instantiated by the supplied testbench without modification:

```
module top (input  logic        clk, reset,
            output logic [31:0] WriteData,
            output logic [31:0] DataAdr,
            output logic        MemWrite);
```

Although WriteData, DataAdr, and MemWrite are conceptually internal signals exchanged between the riscv core and the memory module, they are tapped out as outputs of top so that the testbench can monitor memory transactions and verify that the program writes the expected value to the expected address.

The processor is described hierarchically. The top module instantiates the riscv core and the unified memory module. The riscv module in turn instantiates the controller (which contains the Main FSM, ALU Decoder, and Instruction Decoder as separate submodules) and the datapath (which is built from generic, parameterized building blocks: flopr, flopenr, mux2, mux3, regfile, alu, extend, and adder). All building blocks are written to be reusable across designs and are kept in a single building_blocks.sv file.

4) Testbench and verification methodology

The supplied testbench (riscv_testbench.sv) verifies the processor by monitoring memory writes. Its decision rule is straightforward: on every falling clock edge during which MemWrite is asserted, the testbench inspects the address being written. If a write occurs at address 100 (0x64) with data 25 (0x19), the testbench prints "Simulation succeeded" and stops; if any write occurs at an address other than 96 (the only intermediate write in the test program, produced by sw x7, 84(x3)), it prints "Simulation failed" and stops.

The test program (riscvtest.s) exercises ten different RISC-V instructions — addi, or, and, add, sub, slt, beq, lw, sw, and jal — and is loaded into the unified instruction/data memory at simulation startup using:

```
initial $readmemh("memfile.txt", RAM);
```

inside the memory module.

Before running the simulation, the expected sequence of operations for the first three instructions was predicted manually and recorded in Table 1 (Section 5). This prediction served as a reference during debugging: comparing it against the captured waveforms makes the first cycle of disagreement easy to spot, after which the corresponding control signal or datapath path can be isolated. The waveforms shown in Section 6 are displayed in hexadecimal, in the order requested by the lab handout: clk, reset, PC, Instr, state, SrcA, SrcB, ALUResult, Adr, WriteData, and MemWrite. Auxiliary signals (ALUOut, Result, A, ImmExt) were also added to the wave window to facilitate cycle-by-cycle inspection of the FSM, but they are omitted from the figures included in this report.

The design compiled with no warnings in Questa, and the simulation completed at 725 ns with the expected Simulation succeeded message, indicating that all 21 instructions of the test program executed correctly.

5) Predicted operation

Table 1 lists the expected values of the key processor signals during the first three instructions of the test program (addi x2, x0, 5; addi x3, x0, 12; addi x7, x3, -9). Each addi instruction completes in four cycles (S0 Fetch → S1 Decode → S8 ExecuteI → S7 ALUWB), so 12 entries are recorded. Step numbering follows the lab handout, which begins counting after the two reset cycles.

Step	PC	Instr	State	Result / Notes
3	00	00500113	S0: Fetch	Result = 4 (PC + 4)
4	04	""	S1: Decode	Result = X (OldPC + Imm)
5	04	""	S8: ExecuteI	ALUResult = x0 + 5 = 5
6	04	""	S7: ALUWB	Result = 5 → x2

Step	PC	Instr	State	Result / Notes
7	04	""	S0: Fetch	Result = 8 (PC + 4)
8	08	00C00193	S1: Decode	Result = X (OldPC + Imm)
9	08	""	S8: ExecuteI	ALUResult = x0 + 12 = 12
10	08	""	S7: ALUWB	Result = 12 → x3
11	08	""	S0: Fetch	Result = 12 (PC + 4)
12	0C	FF718393	S1: Decode	Result = X (OldPC + Imm)
13	0C	""	S8: ExecuteI	ALUResult = x3 + (-9) = 3
14	0C	""	S7: ALUWB	Result = 3 → x7

6) Simulation waveforms

The design was simulated in Questa Altera Starter FPGA Edition 2025.2 using the supplied riscv_testbench.sv and the test program loaded from memfile.txt. All required signals were captured and displayed in hexadecimal, in the order specified by the lab handout: clk, reset, PC, Instr, state, SrcA, SrcB, ALUResult, Adr (DataAdr), WriteData, and MemWrite. Auxiliary signals such as ALUOut, Result, A, and ImmExt were also kept in the wave window during debugging but are omitted from the figures shown here for clarity.

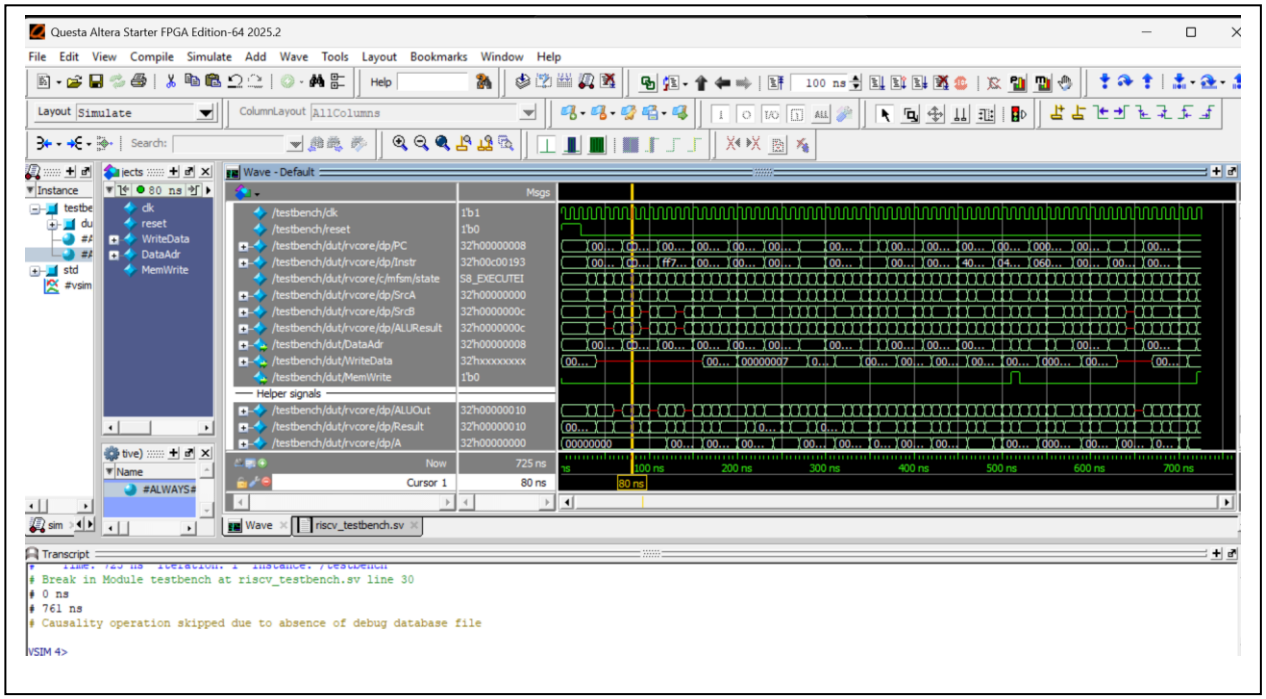

Figure 2. Complete simulation of the multicycle RISC-V processor, from reset until the testbench halts at 725 ns. All required signals are shown in hexadecimal and in the order specified by the lab handout.

Figure 2 shows the entire simulation. After the initial reset pulse (asserted for the first 22 ns), the processor begins fetching and executing the instructions of riscvtest.s sequentially. The state signal transitions through the eleven states of the Main FSM as expected: every instruction starts with S0_Fetch (state = 0) followed by S1_Decode (state = 1), and then branches to instruction-specific states such as S8_ExecuteI for addi, S6_ExecuteR for R-type instructions, S2_MemAdr followed by S3_MemRead/S4_MemWB for lw, S5_MemWrite for sw, S10_BEQ for beq, and S9_JAL for jal. The PC advances by 4 each time the FSM returns to S0_Fetch, except where altered by the taken branch at PC = 0x20 and the jump at PC = 0x40, both of which are clearly visible as discontinuities in the PC waveform. The Instr register, updated only when IRWrite is asserted (i.e., during fetch states), holds each instruction stable for the duration of its execution.

Comparing the early portion of Figure 2 against the predictions in Table 1 confirms that the first three addi instructions execute correctly. At step 5 (cycle reaching S8_ExecuteI of the first addi), SrcA is 0x00000000 (the value of x0) and SrcB is 0x00000005 (the sign-extended immediate), producing ALUResult = 0x00000005, which is then written to x2 during the subsequent S7_ALUWB state. The corresponding writes for x3 = 12 and x7 = 3 follow in the next two addi instructions, exactly as predicted.

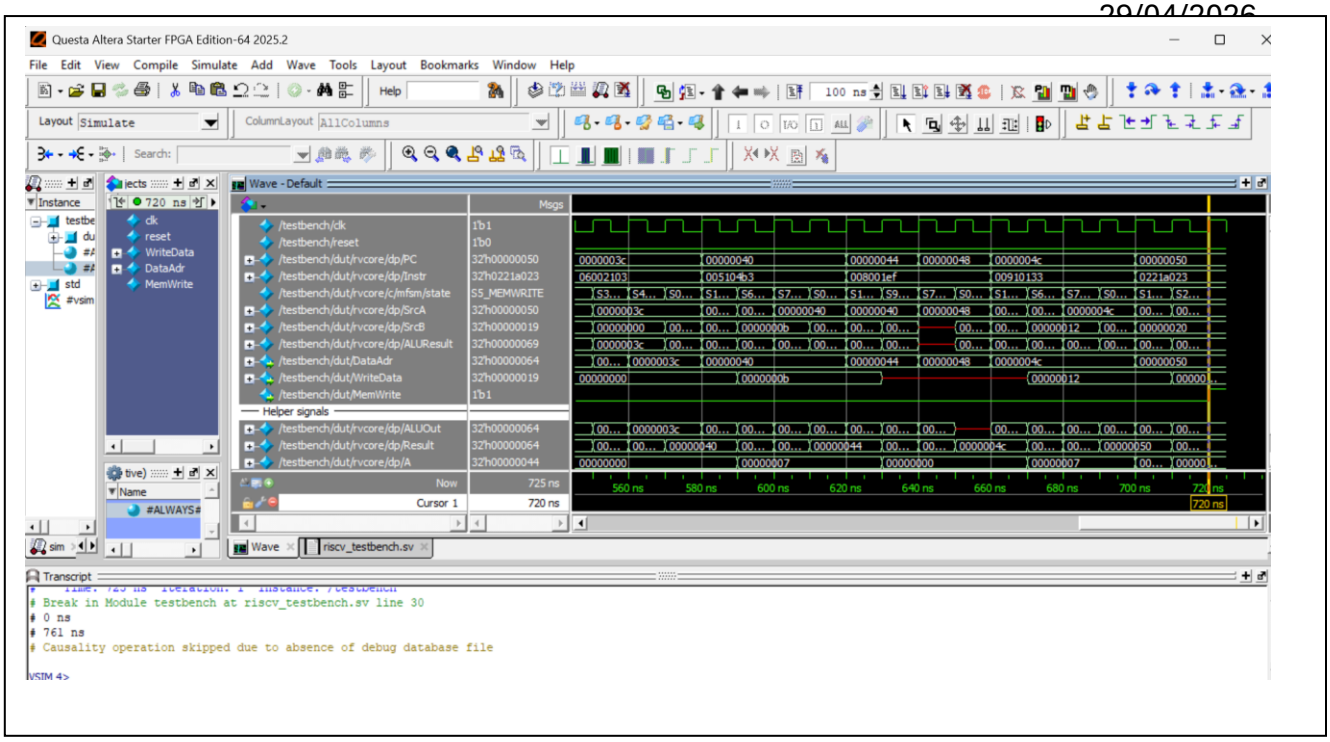

Figure 3. Final memory write of the test program. At this cycle, MemWrite is asserted while DataAdr = 0x00000064 and WriteData = 0x00000019, satisfying the testbench's success condition. The three highlighted signals correspond to the criterion mem[100] = 25.

Figure 3 zooms in on the final MemWrite pulse, generated by the instruction sw x2, 0x20(x3) at PC = 0x4C. At this moment, the datapath drives the address bus with 0x00000064 (= 100 in decimal) and the write data bus with 0x00000019 (= 25 in decimal), while the controller asserts MemWrite. These three signals — highlighted with a red rectangle in the figure — correspond exactly to the success criterion encoded in the testbench: a write of the value 25 to address 100. The testbench detects this condition on the immediately following negative clock edge and prints Simulation succeeded to the transcript before stopping the simulation at 725 ns.

The match between the predicted values in Table 1 and the captured waveforms for the first three instructions, combined with the successful completion of all subsequent instructions, indicates that the controller, the datapath, and the unified memory all behave as designed. The processor passes the supplied testbench without modification.

7) Conclusion

A multicycle RISC-V processor implementing ten instructions (addi, or, and, add, sub, slt, beq, lw, sw, and jal) was successfully designed in SystemVerilog and verified using the testbench supplied with the lab. The design follows the multicycle organization of Chapter 7.4 of Harris and Harris and is partitioned into a top module that wires together a riscv processor core (composed of a controller and a datapath) and a unified memory module. The controller is implemented as an 11-state Moore finite state machine, with the ALU Decoder and Instruction Decoder factored into separate submodules in accordance with the textbook.

The processor passes the supplied testbench. At 725 ns of simulated time, the program writes the value 0x19 (= 25) to address 0x64 (= 100), satisfying the testbench's success condition; the message Simulation succeeded is printed to the transcript. The signal values predicted in Table 1 for the first three instructions agree with the captured waveforms cycle by cycle, confirming that the FSM transitions, ALU operations, and register writeback behave as designed for both S8_ExecuteI and S7_ALUWB states. The remaining instructions of the test program (R-type ALU operations, the taken beq, the sw–lw pair, and the jal) execute correctly without further intervention, indicating that all eleven FSM states are exercised and that the unified instruction/data memory interface is correctly handled by the AdrSrc multiplexer.

The estimated time spent on this lab is approximately **6 hours**, including design, debugging, and report preparation. During initial bring-up, the simulation produced X values on Adr during fetch states, which was traced to the AdrSrc mux not being driven properly when the FSM was in S0_Fetch. Adding the missing assignment in the controller's Moore output table resolved the issue. All SystemVerilog source files, the test program, the simulation script (run.do), and the testbench transcript have been uploaded to the course repository at **https://github.com/aslitas/ELE432-Lab3-Multicycle-RISCV_ASLI** and starred as instructed.